

M07 / PROTECTED ANSWERS

M07 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	4
How to use the protected answers	4
A01 - DE07-01 Verify PostgreSQL connection and database/schema context	4
Expected answer / validation	4
A02 - DE07-02 Tables, data types and explicit DDL	5
Expected answer / validation	5
A03 - DE07-03 Primary keys, foreign keys and constraints	5
Expected answer / validation	5
A04 - DE07-04 Transactions, DML and UPSERT	6
Expected answer / validation	6
A05 - DE07-05 Indexes and EXPLAIN evidence	6
Expected answer / validation	6
A06 - DE07-06 Operational vs analytical modeling and normalization	7
Expected answer / validation	7
A07 - DE07-07 Facts, dimensions and star-schema grain	7
Expected answer / validation	7
A08 - DE07-08 Surrogate keys and referential integrity	8
Expected answer / validation	8
A09 - DE07-09 Slowly Changing Dimension Type 2 history	8
Expected answer / validation	8
A10 - DE07-10 Build and validate a small analytical database	9

Expected answer / validation 9

A11 - DE07-11 | Bulk loading with COPY / staging boundaries 10

Expected answer / validation 10

A12 - DE07-12 | Schemas, roles and least-privilege boundaries 10

Expected answer / validation 10

A13 - DE07-13 | Schema change, backup/restore and handoff validation 11

Expected answer / validation 11

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

PROTECTED TEST ANSWERS

Use this file to validate and repair, not to erase your first attempt. Compare the expected control with what you actually observed, close this guide, then solve a changed retry if the competency was weak.

Coverage: 13 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

How to use the protected answers

Open only the lesson you attempted. The guide gives the expected control/behavior, not permission to copy. If you used it, close it and perform a changed retry before marking the lesson mastered.

A01 - DE07-01 | Verify PostgreSQL connection and database/schema context

Expected answer / validation

What strong reasoning looks like A strong attempt proves context from the active PostgreSQL session, not from the pgAdmin browser tree or your memory. You should be able to distinguish server/cluster, database, schema and table. If `current_database()` is wrong, reconnect before DDL. If `to_regclass('table')` and `to_regclass('schema.table')` differ, explain that unqualified lookup is search-path dependent. A common weak answer says "schema is another database"; repair that language before retrying. The changed retry should use two schemas with the same relation name so the ambiguity becomes visible.

- Repair questions
- What exactly was the row/object grain?
- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Use raw/serving schemas with the same table name; predict three `search_path` resolutions before querying.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A02 - DE07-02 | Tables, data types and explicit DDL

Expected answer / validation

What strong reasoning looks like Review your types by business meaning. Identifiers are not numbers merely because they contain digits. Exact currency-like values should not silently become approximate floats. A date is different from an event instant. A DEFAULT only supplies a value when a column is omitted; it does not mean "repair missing source data." Compare your intended DDL to `information_schema.columns` so you prove what PostgreSQL actually created. If you used one generated surrogate key, you should still preserve a separate business identifier and its uniqueness rule.

- Repair questions
- What exactly was the row/object grain?
- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the
- command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Design receipt header/line DDL with leading-zero business ID, exact money, date and one

generated surrogate.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A03 - DE07-03 | Primary keys, foreign keys and constraints

Expected answer / validation

What strong reasoning looks like Your constraint tests should include successful and rejected rows. Preserve at least one PK/UNIQUE, FK, CHECK and NOT NULL failure where relevant. If a CHECK alone was expected to reject NULL, revisit PostgreSQL CHECK semantics: an expression that becomes NULL is not false, so NOT NULL may also be required. A foreign key proves parent-key existence, not "the whole relationship is correct." The changed retry should make a composite-key need obvious by allowing line 1 in more than one parent document.

- Repair questions
- What exactly was the row/object grain?
- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the
- command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Receipt line composite key plus missing-parent/blank-text failure set; state what each rule still cannot prove.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A04 - DE07-04 | Transactions, DML and UPSERT

Expected answer / validation

What strong reasoning looks like For transaction mastery, show state before the transaction, changed state inside it, and restored/committed state afterward. If an error caused "current transaction is aborted," the first error is the root cause; rollback or rollback to a savepoint restores usable transaction state. For UPSERT, name the conflict key and the row grain it represents. Exact rerun should converge on the same current-state row. If your conflict key overwrites history rows, the model is wrong even when ON CONFLICT executes perfectly.

- Repair questions
- What exactly was the row/object grain?
- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Supplier current-state upsert + intentional two-write failure; explain why SCD2 cannot use overwrite semantics.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A05 - DE07-05 | Indexes and EXPLAIN evidence

Expected answer / validation

What strong reasoning looks like A good index explanation begins with the query pattern and selectivity, not the index command. On tiny tables, Seq Scan can be the rational plan. EXPLAIN cost is planner cost, not milliseconds. EXPLAIN ANALYZE executes the statement. Compare the same business result before/after any index change. If you changed filters while comparing plans, your experiment is invalid. For a composite index, explain why leading-column order matches the expected query pattern. One documented downside - write/storage maintenance - should appear in your defense.

- Repair questions
- What exactly was the row/object grain?
- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the

- command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Create dominant-vs-rare hub distribution and predict plan/index usefulness for each predicate.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A06 - DE07-06 | Operational vs analytical modeling and normalization

Expected answer / validation

What strong reasoning looks like A strong model separates current operational facts from analytical reporting needs. In a flat order-line table, repeated customer attributes create update anomalies. 1NF/2NF/3NF language is useful only if you can connect it to practical dependencies. Normalizing customers/orders/order_items does not automatically preserve historical region. Your star model should be justified by analytical grain and history, not by saying "star schema is faster." The changed retry should use a different domain and expose one concrete anomaly in the flat design.

- Repair questions
- What exactly was the row/object grain?
- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the
- command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Supplier receipt flat export -> normalized operational model -> star; identify one update anomaly and one history question.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A07 - DE07-07 | Facts, dimensions and star-schema grain

Expected answer / validation

What strong reasoning looks like Fact grain must be one unambiguous sentence. For the supplied model it is one completed order line. The core source controls are 8 completed lines, 5 completed orders and 1780.00. Repeating an order-level total/fee on each line creates fan-out even though each repeated value is individually correct. Region controls under the SCD2 history are East 660, South 600 and West 520. If the grand total is 1780 but regions are wrong, investigate temporal attribution rather than the fact amount.

- Repair questions
- What exactly was the row/object grain?

- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Receipt-line fact with one header freight amount; choose a safe modeling strategy and reconcile changed controls.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A08 - DE07-08 | Surrogate keys and referential integrity

Expected answer / validation

What strong reasoning looks like The natural/business key answers which entity; the surrogate key answers which warehouse row/version. A surrogate primary key does not prevent two rows representing the same business entity unless a separate business/history rule does. In SCD2, multiple surrogate rows for one customer are intentional, so `UNIQUE(customer_id)` across the whole history is wrong. The fact stores the historical surrogate key, but the as-of join/load logic must still prove that the selected version was valid for the event date.

- Repair questions
- What exactly was the row/object grain?
- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Product SCD2 dimension where one business product legitimately has several surrogate versions.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A09 - DE07-09 | Slowly Changing Dimension Type 2 history

Expected answer / validation

What strong reasoning looks like Use half-open intervals: `valid_from <= event_date < valid_to`. C001 should map O101 to East and O102 to South. Prove exactly one current row, every `valid_from < valid_to`, and no overlapping windows. If both orders report South, you joined to current state. If a fact duplicates, overlapping dimension windows are a likely cause. SCD2 is justified only for attributes whose historical attribution matters; do not create history merely because it sounds advanced.

- Repair questions

- What exactly was the row/object grain?
- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Supplier risk Low->High; compare current-risk report with risk-at-receipt report.

Move-on rule Close this review. Reproduce the changed task independently, save the observed result/error and validation, then explain the concept without notes. If the changed task still depends on this page, remain in repair rather than marking mastery.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A10 - DE07-10 | Build and validate a small analytical database

Expected answer / validation

What strong reasoning looks like End-to-end validation should use redundant controls. The supplied source has 9 lines / 2280.00 total; the Completed population has 8 lines / 1780.00 / 5 distinct orders. The fact must match the filtered population, not the unfiltered source. A correct amount with wrong row count can hide offsetting duplicate/missing rows. A correct row count with wrong amount can hide corrupted measures. Your architecture defense should explain grain, keys, SCD timing, constraints, remaining validation and recovery evidence.

- Repair questions
- What exactly was the row/object grain?
- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Fresh receipt/supplier analytical database with different keys, history attribute and reconciliation controls.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A11 - DE07-11 | Bulk loading with COPY / staging boundaries

Expected answer / validation

What strong reasoning looks like Do not confuse SQL COPY with psql \copy. Server-side COPY sees server paths/permissions; \copy makes the psql client read the file. Raw staging should preserve inspectable source values before typed promotion. For customer_stage_bad.csv, source rows=5, accepted=2, rejected/duplicate=3. Keep reject reasons. PostgreSQL 18 has COPY error-handling options, but syntax-level acceptance does not replace business validation for blank region or duplicate business keys.

- Repair questions
- What exactly was the row/object grain?
- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the
- command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Fresh dirty receipt CSV with malformed amount, duplicate line key and valid late row; preserve rejects.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A12 - DE07-12 | Schemas, roles and least-privilege boundaries

Expected answer / validation

What strong reasoning looks like A least-privilege design separates ownership/admin, routine writes and reads. Schema USAGE and table SELECT/INSERT are separate layers. Test a denial that the policy intends - for example analyst INSERT should fail. If you cannot safely create roles on your environment, label the plan as pending instead of fabricating screenshots. Remember that grants on current tables do not automatically define privileges for future objects; default privileges are tied to the role creating those future objects.

- Repair questions
- What exactly was the row/object grain?
- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the
- command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Reporter role can read serving only; cannot see staging/write/create. List exact minimum grants.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A13 - DE07-13 | Schema change, backup/restore and handoff validation

Expected answer / validation

What strong reasoning looks like A dump file is not recovery proof. pg_dump exports one database; cluster-global roles/tablespaces are separate concerns. Restore into a different clean database, then rerun the same object/data/business controls. A staged migration should separate adding a structure from backfill/validation and final constraint enforcement where appropriate. Handoff should remove absolute personal paths and disclose which tests were actually executed versus still native-runtime pending.

- Repair questions
- What exactly was the row/object grain?
- Which assumption did PostgreSQL enforce, and which one remained a pipeline/business rule?
- What observable result/error/plan proves the behavior instead of merely showing the
- command text?
- Did your control use the exact same population as the result you are validating?

Changed retry Fresh model dump/restore into a new target; eliminate one hidden absolute-path or local-state dependency.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.